Review the assignment due date

实验一：自动驾驶车辆执行器

实践日期： 2026-4-5　　实践课程：Rust 编程语言

学号：10235101495　　姓名：李贤达

本实验围绕 `Executor` 组件展开，目标是模拟自动驾驶车辆执行器对基础控制指令的处理过程，并通过测试先行的方式，确保组件行为符合预期。

1. 实验目标

1. 提供初始化接口，设置车辆初始位置和朝向。
2. 支持三种基础指令

- `M`：沿当前朝向前进一步
- `L`：左转 90 度，位置不变
- `R`：右转 90 度，位置不变

3. 提供查询接口，返回当前车辆坐标和朝向

并包含以下约束条件：

- `x` 和 `y` 使用 `i32`
- `heading` 使用 `char`，范围为 `E`、`S`、`W`、`N`
- 不考虑整数溢出
- 调用者保证输入参数合法

2. 项目结构

```
first_program/
├── Cargo.toml                  # workspace 配置
├── README.md
└── executor/
    ├── Cargo.toml              # executor crate
    ├── src/
    │   ├── lib.rs              # 对外暴露公共接口
    │   ├── executor.rs         # Executor 与 Pose 的核心实现
    │   └── main.rs             # 命令行交互入口
    └── tests/
        └── executor_test.rs    # 集成测试文件
```

这个工程采用了 crate 的组织方式：

- `executor` 是真正实现功能模块的 crate
- `tests/executor_test.rs` 则是通过公共接口测试组件行为

3. 接口设计

3.1 Pose

结构体 `Pose` 用于描述车辆当前位置以及朝向：

```
pub struct Pose {
    pub x: i32,
    pub y: i32,
    pub heading: char,
}
```

3.2 Executor

`Executor` 表示执行器组件，内部保存当前车辆状态：

```
pub struct Executor {
    pose: Pose,
}
```

Executor提供对外接口：

- `Executor::with_pose(pose)`：按指定`Pose`初始化
- `Executor::new(pose)`：与 `with_pose` 语义一致，便于测试编写
- `execute(&mut self, cmds: &str)`：执行一串控制指令
- `query(&self) -> Pose`：查询当前`Pose`
- `Default for Executor`：创建默认状态的执行器

4. 命名规范性

本项目中的命名体现如下：

- 类型名使用大驼峰：`Executor`、`Pose`
- 模块名、函数名、方法名使用蛇形：`move_forward`、`turn_left`、`turn_right`
- 测试函数名使用 `should_..._given_...` 格式

例如：

```
fn should_return_x_plus_1_given_command_is_m_and_facing_is_e()
```

该命名可以直观地表达以下三层信息：

- 预期结果：`should_return_x_plus_1`
- 触发条件：`given_command_is_m`
- 前置状态：`and_facing_is_e`

测试失败时，只看函数名就能知道哪些功能实现存在问题。

5. 语法说明

5.1 引入被测试对象

```
use executor::{Executor, Pose};
```

`use` 语句用于引入被测试对象，方便在测试函数中直接使用。

5.2 使用模块对测试分组

```
mod move_tests {
    use super::*;
}
```

作用：

- 用 `mod` 按功能组织测试
- `move_tests`、`turn_left_tests`、`turn_right_tests` 能清晰表达分组目的
- `use super::*;` 表示把外层已经引入的名字带进当前测试模块中

5.3 `#[test]` 属性

```
#[test]
fn should_return_x_plus_1_given_command_is_m_and_facing_is_e() {
    // ...
}
```

`#[test]` 是 Rust 的测试标记，其后面的函数要求必须为无参数函数且函数内部通过断言判断是否通过。同时，带有这个属性的函数会在 `cargo test` 时被测试框架自动发现和执行。

5.4 Given-When-Then 写法

测试函数体按 Given-When-Then 三段式组织要求：

```
#[test]
fn should_return_x_plus_1_given_command_is_m_and_facing_is_e() {
    // given
    let original_pose = Pose::new(0, 0, 'E');
    let mut executor = Executor::new(original_pose);

    // when
    executor.execute("M");
```

```
    // then
    let expected_pose = Pose::new(1, 0, 'E');
    assert_eq!(expected_pose, executor.query());
}
```

含义如下：

- Given：准备测试数据和初始环境
- When：执行被测动作
- Then：断言结果是否符合预期

5.5 `assert_eq!` 断言宏

```
assert_eq!(expected_pose, executor.query());
```

测试用例中常用断言宏，用于比较期望值和实际值是否相等。

6. 测试设计原则

6.1 FIRST 原则

Fast

测试要快。
测试执行时间短，便于频繁运行。

Isolated

测试要独立。
每个测试都会自己创建新的 `Pose` 和 `Executor`，互不共享状态，因此测试可以并行执行，不会互相影响。

Self-Validating

测试要自验。
本项目通过 `assert_eq!` 自动判断测试是否成功。

Timely

测试要及时。
在开发过程中及时编写测试，能够更早的发现代码中的潜在漏洞，避免后期大规模重构。

7. 当前测试覆盖内容

本项目已经覆盖以下场景：

- 移动指令 `M` 在四个朝向下的行为
- 左转指令 `L` 在四个朝向下的行为
- 右转指令 `R` 在四个朝向下的行为

- 命令串 `MLMMRM` 的集成行为
- 默认状态下的行为

8. 总结

这个实验的重点不只是“把功能写出来”，而是通过一个很小的业务案例练习高质量编程的基本习惯：

- 先澄清需求，再设计接口
- 用清晰的命名表达业务意图
- 用小而独立的测试建立防护网
- 使用 Rust 的模块系统、trait、模式匹配和断言宏，把代码写得可读、可测、可维护

如果把 `executor/src/executor.rs` 看作“业务实现”，那么 `executor/tests/executor_test.rs` 就是这份业务需求最直接的可执行文档。